

You

You are an expert senior game developer,
you are given a task to fix the bugs found and reported below
fix the Task 1 now and only change whatever is mentioned below only,
any further features and improvements will be done later
follow and do the current task with minimal changes to structure and unwanted changes
give codes directly to copy paste in blocks

The current task is:

Task 1:

The basket is supposed to catch an object only once it has actually fallen down to basket height. In the current build, `is_caught` (in `game/collision.py`) only checks whether the object's x position is within the basket's left/right edges - it never checks height at all. That means an object can score the instant it spawns at the top of the screen, as soon as the basket happens to be under its x position, however far away it still is. Separately, the loop in `update()` (in `game/game_engine.py`) that checks for catches removes objects from the same list it's iterating over - a classic Python mistake that can silently skip checking an object on a given frame when two catchable objects are next to each other in the list. Fix both: catches should require the object to actually be at basket height, and the catch-checking loop should not skip entries.

CODE FILES

collision.py

```
"""
collision: figures out whether a falling object is within the basket.
"""

def is_caught(basket_rect, obj):
    return basket_rect.left <= obj.x <= basket_rect.right
```

game_engine.py

```
"""
GameEngine: owns the basket and all falling objects.

Starter version: basket movement and spawning both work at a basic
level (Tasks 2 and 3 ask you to improve them), there's no speed boost
yet (Task 4 builds it from scratch), and catch detection has two
known bugs (see game/collision.py and the catch-checking loop below)
that Task 1 asks you to fix.
"""

import random
import pygame

from game.basket import Basket
from game.falling_object import FallingObject
from game.collision import is_caught
from game.renderer import WIDTH, HEIGHT
```

```
SPAWN_INTERVAL_FRAMES = 50
MAX_MISSES = 5
```

```
class GameEngine:
    def __init__(self):
        self.basket = Basket(x=WIDTH / 2, y=HEIGHT - 30)
        self.objects = []
        self.frames_until_spawn = 0
        self.score = 0
        self.misses = 0
        self.game_over = False

    def _spawn_object(self):
        x = random.randint(20, WIDTH - 20)
        self.objects.append(FallingObject(x=x, y=-14, speed=3))

    def handle_input(self, keys_pressed):
        if self.game_over:
            return
        if keys_pressed[pygame.K_LEFT]:
            self.basket.x -= self.basket.speed
        if keys_pressed[pygame.K_RIGHT]:
            self.basket.x += self.basket.speed
        # Boundary handling: only clamps against the screen edges, not
        # accounting for the basket's own width - it can hang half off
        # either side of the screen.
        self.basket.x = max(0, min(WIDTH, self.basket.x))

    def handle_keydown(self, key):
        if self.game_over and key == pygame.K_r:
            self.__init__()

    def update(self):
        if self.game_over:
            return

        self.frames_until_spawn -= 1
        if self.frames_until_spawn <= 0:
            self._spawn_object()
            self.frames_until_spawn = SPAWN_INTERVAL_FRAMES

        for obj in self.objects:
            obj.update()

        basket_rect = self.basket.get_rect()
        for obj in self.objects: # BUG: mutating this list while iterating over it
            if is_caught(basket_rect, obj):
                self.score += 1
                self.objects.remove(obj)

        missed = [o for o in self.objects if o.is_past_bottom(HEIGHT)]
        if missed:
            self.objects = [o for o in self.objects if not o.is_past_bottom(HEIGHT)]
            self.misses += len(missed)
            if self.misses >= MAX_MISSES:
                self.game_over = True
```

```

def draw(self, surface, font):
    from game import renderer
    renderer.draw_scene(surface, self.basket, self.objects)
    renderer.draw_text(surface, font, f"Score: {self.score}", (10, 10))
    renderer.draw_text(surface, font, f"Misses: {self.misses}/{MAX_MISSES}", (10,
        36))

    if self.game_over:
        renderer.draw_banner(surface, font, f"Game Over! Final score: {self.score}.
            Press R to restart.")

```

ChatGPT

Today 12:11 PM

Replace game/collision.py with:

```

"""
collision: figures out whether a falling object is within the basket.
"""

def is_caught(basket_rect, obj):
    return (
        basket_rect.left <= obj.x <= basket_rect.right
        and basket_rect.top <= obj.y <= basket_rect.bottom
    )

```

In game/game_engine.py, replace only the catch-checking block inside update() with:

```

basket_rect = self.basket.get_rect()
for obj in self.objects[:]:
    if is_caught(basket_rect, obj):
        self.score += 1
        self.objects.remove(obj)

```

Catches now require both horizontal and vertical alignment. Iterating over a copy prevents removals from skipping adjacent objects.

You

Cool, now its working well after the fixes

Now we have the next task

Task 2:

The basket can currently be moved half off either edge of the screen, since its boundary check doesn't account for its own width. Fix the boundary handling so the basket always stays fully on screen, and make sure movement feels smooth and responsive even when a direction key is held down continuously.

Using the same file as last time, assuming the updated code

ChatGPT

In `game/game_engine.py`, replace only `handle_input()` with:

```
def handle_input(self, keys_pressed):
    if self.game_over:
        return

    if keys_pressed[pygame.K_LEFT]:
        self.basket.x -= self.basket.speed
    if keys_pressed[pygame.K_RIGHT]:
        self.basket.x += self.basket.speed

    half_width = self.basket.get_rect().width / 2
    self.basket.x = max(
        half_width,
        min(WIDTH - half_width, self.basket.x),
    )
```

The boundary check now accounts for the basket's width. Keep calling `handle_input()` with `pygame.key.get_pressed()` every frame to preserve continuous movement while a direction key is held down.

You

its working now,
now for the next task 3

Improve the spawning system so objects appear at varied horizontal positions and at varied intervals, rather than a fixed interval with a plain random x each time. Make sure objects never spawn outside the playable area, avoid repeatedly spawning in the same spot, and cap how many objects can be on screen at once so the game never becomes unplayable.

ChatGPT

Update these four parts of `game/game_engine.py`.

Replace the existing constants with:

```
MIN_SPAWN_INTERVAL_FRAMES = 30
MAX_SPAWN_INTERVAL_FRAMES = 70
MAX_OBJECTS = 5
SPAWN_MARGIN = 20
MIN_SPAWN_DISTANCE = 60
MAX_MISSES = 5
```

Add this line inside `__init__()`, after `self.frames_until_spawn = 0`:

```
self.last_spawn_x = None
```

Replace `_spawn_object()` with:

```
def _spawn_object(self):
    if len(self.objects) >= MAX_OBJECTS:
        return

    min_x = SPAWN_MARGIN
    max_x = WIDTH - SPAWN_MARGIN
    spawn_ranges = [(min_x, max_x)]

    if self.last_spawn_x is not None:
        spawn_ranges = []

        left_end = self.last_spawn_x - MIN_SPAWN_DISTANCE
        right_start = self.last_spawn_x + MIN_SPAWN_DISTANCE

        if left_end >= min_x:
            spawn_ranges.append((min_x, left_end))

        if right_start <= max_x:
            spawn_ranges.append((right_start, max_x))

    if spawn_ranges:
        start, end = random.choice(spawn_ranges)
        x = random.randint(start, end)
    else:
        # On narrow screens, use the furthest available edge.
        x = max(
            (min_x, max_x),
            key=lambda position: abs(position - self.last_spawn_x),
        )

    self.objects.append(FallingObject(x=x, y=-14, speed=3))
    self.last_spawn_x = x
```

Inside `update()`, replace the existing spawn countdown block with:

```
self.frames_until_spawn -= 1
if self.frames_until_spawn <= 0:
    self._spawn_object()
    self.frames_until_spawn = random.randint(
        MIN_SPAWN_INTERVAL_FRAMES,
        MAX_SPAWN_INTERVAL_FRAMES,
    )
```

This varies spawn attempts between 30–70 frames, caps active objects at 5, and keeps successive spawn positions at least 60 pixels apart when space permits, using the existing 20-pixel edge margin.

You

Today 3:59 PM

thats done

now for task 4

Add a speed-boost mechanic the player can activate during play that temporarily increases the basket's movement speed. Show a clear indication while the boost is active, and have the basket automatically return to normal speed once it expires.

Press Space for double movement speed for 3 seconds, with an on-screen countdown.

ChatGPT

Make these changes in `game/game_engine.py`.

Add these constants alongside the existing constants:

```
BOOST_MULTIPLIER = 2
BOOST_DURATION_MS = 3000
```

Add this line inside `__init__()`:

```
self.boost_end_time = 0
```

Replace `handle_input()` with:

```
def handle_input(self, keys_pressed):
    if self.game_over:
        return

    movement_speed = self.basket.speed
    if pygame.time.get_ticks() < self.boost_end_time:
        movement_speed *= BOOST_MULTIPLIER

    if keys_pressed[pygame.K_LEFT]:
        self.basket.x -= movement_speed
    if keys_pressed[pygame.K_RIGHT]:
        self.basket.x += movement_speed

    half_width = self.basket.get_rect().width / 2
    self.basket.x = max(
        half_width,
        min(WIDTH - half_width, self.basket.x),
    )
```

Replace `handle_keydown()` with:

```
def handle_keydown(self, key):
    if self.game_over:
        if key == pygame.K_r:
            self.__init__()
        return

    if key == pygame.K_SPACE:
        now = pygame.time.get_ticks()

        # An active boost cannot be extended or stacked.
        if now >= self.boost_end_time:
            self.boost_end_time = now + BOOST_DURATION_MS
```

Inside `draw()`, add this immediately after the existing Misses text and before the game-over check:

```
if not self.game_over:
    boost_remaining = (
        self.boost_end_time - pygame.time.get_ticks()
    )

    if boost_remaining > 0:
        boost_text = (
            f"BOOST ACTIVE: {boost_remaining / 1000:.1f}s"
        )
    else:
        boost_text = "Boost: SPACE"

    renderer.draw_text(
        surface, font, boost_text, (10, 62)
    )
```

Movement automatically returns to normal when the timer expires. Space can then activate another boost.